

Name: - Shubham Das
Date: - 25/09/2025

S. A. P I. D: - 590033500
Programme: - B. TECH (CSE)

Assignment: - Write a C program to perform the following menu-driven operations using separate functions:

1. Get two variable length 2D arrays and prove that matrix addition is commutative.
2. Get two variable length 2D arrays and prove that matrix multiplication is not commutative.
3. Exit

Part 1: Thorough understanding of the problem statement

Input: -

1. Menu *choice* (Variable: *int*, Unit: unitless)
2. Matrix dimensions *rows*, *cols*, or *n* (Variables: *int*, Unit: unitless)
3. Matrix elements for Array A and Array B (Variables: *int*, Unit: scalar values)

Output: -

1. Computed matrices for $A + B$, $B + A$, $A \times B$, $B \times A$ (Variable: *int* 2D arrays).
2. Boolean comparison results proving commutativity or non-commutativity.

Formula: -

1. Matrix Addition: $Result[i][j] = A[i][j] + B[i][j];$
2. Matrix Multiplication: $Result[i][j] += A[i][k] * B[k][j];$
3. Commutativity check: $if (Result1[i][j] != Result2[i][j]) \{ isCommutative = false; \}$

Part 2: - Writing, compiling and evaluating the algorithm

Step 1: - Start

Step 2: - Declare modular functions leveraging C99 Variable Length Arrays (VLAs) to dynamically pass matrix dimensions and arrays: *read_matrix*, *print_matrix*, *add_matrix*, *multiply_matrices* and *compare_matrices*.

Step 3: - Initiate a *do-while* loop in *main()* to display the menu and capture user *choice*.

Step 4 (Case 1- Addition):

- 1: - Prompt for *rows* and *cols*.
- 2: - Declare VLAs $A[rows][cols]$, $B[rows][cols]$, $AB[rows][cols]$, $BA[rows][cols]$.

- 3: - Compute $AB = A + B$ and $BA = B + A$.
- 4: - Compare AB and BA . If all elements match, output that matrix addition is commutative.

Step 5: - (Case 2 - Multiplication);

- 1: - Prompt for square matrix dimension n (required to ensure both $A \times B$ and $B \times A$ yield valid matrices of the exact same dimensions for direct comparison).
- 2: - Declare VLAs $A[n][n]$, $B[n][n]$, $AB[n][n]$, $BA[n][n]$.
- 3: - Read elements for A and B .
- 4: - Compute $AB = A * B$ and $BA = B * A$.
- 5: - Compare AB and BA . If any element differs, output that matrix multiplication is not commutative.
- 6: - (Case 3): Exit the programme.
- 7: - Stop.

Part 3: - Writing and compiling the program

GNU nano 9.0

```
#include <stdio.h>
#include <stdbool.h>

// Function prototypes using C99 Variable Length Arrays (VLAs)
void read_matrix(int rows, int cols, int mat[rows][cols], const char* name);
void print_matrix(int rows, int cols, int mat[rows][cols], const char* name);
void add_matrices(int rows, int cols, int A[rows][cols], int B[rows][cols], int res[rows][cols]);
void multiply_matrices(int n, int A[n][n], int B[n][n], int res[n][n]);
bool compare_matrices(int rows, int cols, int A[rows][cols], int B[rows][cols]);
void prove_addition_commutativity(void);
void prove_multiplication_noncommutativity(void);

int main() {
    int choice;

    do {
        printf("\n===== MATRIX PROPERTIES MENU =====\n");
        printf("1. Prove Matrix Addition is Commutative\n");
        printf("2. Prove Matrix Multiplication is NOT Commutative\n");
        printf("3. Exit\n");
        printf("Enter your choice (1-3): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                prove_addition_commutativity();
                break;
            case 2:
                prove_multiplication_noncommutativity();
                break;
            case 3:
                printf("Exiting program. Goodbye!\n");
                break;
            default:
                printf("Error: Invalid choice. Please select 1, 2, or 3.\n");
        }
    } while (choice != 3);

    return 0;
}

// Helper Functions
void read_matrix(int rows, int cols, int mat[rows][cols], const char* name) {
    printf("Enter elements for %d x %d Matrix %s:\n", rows, cols, name);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &mat[i][j]);
        }
    }
}
```

GNU nano 9.0

```
}  
}  
  
void print_matrix(int rows, int cols, int mat[rows][cols], const char* name) {  
    printf("Matrix %s:\n", name);  
    for (int i = 0; i < rows; i++) {  
        for (int j = 0; j < cols; j++) {  
            printf("%4d ", mat[i][j]);  
        }  
        printf("\n");  
    }  
}  
  
void add_matrices(int rows, int cols, int A[rows][cols], int B[rows][cols], int res[rows][cols]) {  
    for (int i = 0; i < rows; i++) {  
        for (int j = 0; j < cols; j++) {  
            res[i][j] = A[i][j] + B[i][j];  
        }  
    }  
}  
  
void multiply_matrices(int n, int A[n][n], int B[n][n], int res[n][n]) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            res[i][j] = 0;  
            for (int k = 0; k < n; k++) {  
                res[i][j] += A[i][k] * B[k][j];  
            }  
        }  
    }  
}  
  
bool compare_matrices(int rows, int cols, int A[rows][cols], int B[rows][cols]) {  
    for (int i = 0; i < rows; i++) {  
        for (int j = 0; j < cols; j++) {  
            if (A[i][j] != B[i][j]) return false;  
        }  
    }  
    return true;  
}  
  
// Logic Functions  
void prove_addition_commutativity(void) {  
    int rows, cols;  
    printf("\n--- Prove Addition (A + B = B + A) ---\n");  
    printf("Enter number of rows and columns: ");  
    scanf("%d %d", &rows, &cols);
```

```

GNU nano 9.0 matrix_proof.c *
if (rows <= 0 || cols <= 0) {
    printf("Error: Dimensions must be greater than zero.\n");
    return;
}

int A[rows][cols], B[rows][cols], AB[rows][cols], BA[rows][cols];

read_matrix(rows, cols, A, "A");
read_matrix(rows, cols, B, "B");

add_matrices(rows, cols, A, B, AB); // Compute A + B
add_matrices(rows, cols, B, A, BA); // Compute B + A

print_matrix(rows, cols, AB, "(A + B)");
print_matrix(rows, cols, BA, "(B + A)");

if (compare_matrices(rows, cols, AB, BA)) {
    printf("\nCONCLUSION: Since (A + B) == (B + A), matrix addition IS commutative.\n");
} else {
    printf("\nCONCLUSION: Matrices do not match (Unexpected error).\n");
}
}

void prove_multiplication_noncommutativity(void) {
    int n;
    printf("\n--- Prove Multiplication (A * B != B * A) ---\n");
    printf("Enter dimension N for NxN square matrices (required for valid AB and BA comparisons): ");
    scanf("%d", &n);

    if (n <= 1) {
        printf("Error: Dimension must be 2 or greater to prove non-commutativity effectively.\n");
        return;
    }

    int A[n][n], B[n][n], AB[n][n], BA[n][n];

    read_matrix(n, n, A, "A");
    read_matrix(n, n, B, "B");

    multiply_matrices(n, A, B, AB); // Compute A * B
    multiply_matrices(n, B, A, BA); // Compute B * A

    print_matrix(n, n, AB, "(A * B)");
    print_matrix(n, n, BA, "(B * A)");

    if (!compare_matrices(n, n, AB, BA)) {
        printf("\nCONCLUSION: Since (A * B) != (B * A), matrix multiplication IS NOT commutative.\n");
    } else {
        printf("\nCONCLUSION: Matrices accidentally matched. (Try using random non-identity/non-zero values to prove non-commutativity).\n");
    }
}

```

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

Step 1: - Open the terminal

Step 2: - Create the source code

Step 3: - Write and Save Code

Step 4: - Compile the program

Step 5: - Run the Binary Executable programme

```
(kali㉿kali)-[~]  
$ nano matrix_proof.c  
  
(kali㉿kali)-[~]  
$ gcc matrix_proof.c -o matrix_proof  
  
(kali㉿kali)-[~]  
$ ./matrix_proof
```

```
(kali㉿kali)-[~]  
$ ./matrix_proof
```

```
===== MATRIX PROPERTIES MENU =====
```

1. Prove Matrix Addition is Commutative
2. Prove Matrix Multiplication is NOT Commutative
3. Exit

Enter your choice (1-3): 1

--- Prove Addition ($A + B = B + A$) ---

Enter number of rows and columns: 2 2

Enter elements for 2 x 2 Matrix A:

1 2

3 4

Enter elements for 2 x 2 Matrix B:

5 6

7 8

Matrix ($A + B$):

6 8

10 12

Matrix ($B + A$):

6 8

10 12

CONCLUSION: Since $(A + B) == (B + A)$, matrix addition IS commutative.

```
===== MATRIX PROPERTIES MENU =====
```

1. Prove Matrix Addition is Commutative
2. Prove Matrix Multiplication is NOT Commutative
3. Exit

Enter your choice (1-3): 2

--- Prove Multiplication ($A * B != B * A$) ---

Enter dimension N for NxN square matrices (required for valid AB and BA comparisons): 2

Enter elements for 2 x 2 Matrix A:

1 2

3 4

Enter elements for 2 x 2 Matrix B:

5 6

7 8

Matrix ($A * B$):

19 22

43 50

Matrix ($B * A$):

23 34

31 46

CONCLUSION: Since $(A * B) != (B * A)$, matrix multiplication IS NOT commutative.

```
===== MATRIX PROPERTIES MENU =====
```

1. Prove Matrix Addition is Commutative

```

7 8
Matrix (A * B):
19 22
43 50
Matrix (B * A):
23 34
31 46

CONCLUSION: Since (A * B) != (B * A), matrix multiplication IS NOT commutative.

===== MATRIX PROPERTIES MENU =====
1. Prove Matrix Addition is Commutative
2. Prove Matrix Multiplication is NOT Commutative
3. Exit
Enter your choice (1-3): 2

--- Prove Multiplication (A * B != B * A) ---
Enter dimension N for NxN square matrices (required for valid AB and BA comparisons): 2
Enter elements for 2 x 2 Matrix A:
1 0
0 1
Enter elements for 2 x 2 Matrix B:
1 0
0 1
Matrix (A * B):
1 0
0 1
Matrix (B * A):
1 0
0 1

CONCLUSION: Matrices accidentally matched. (Try using random non-identity/non-zero values to prove non-commutativity).

===== MATRIX PROPERTIES MENU =====
1. Prove Matrix Addition is Commutative
2. Prove Matrix Multiplication is NOT Commutative
3. Exit
Enter your choice (1-3): 2

--- Prove Multiplication (A * B != B * A) ---
Enter dimension N for NxN square matrices (required for valid AB and BA comparisons): -1
Error: Dimension must be 2 or greater to prove non-commutativity effectively.

===== MATRIX PROPERTIES MENU =====
1. Prove Matrix Addition is Commutative
2. Prove Matrix Multiplication is NOT Commutative
3. Exit
Enter your choice (1-3): 3
Exiting program. Goodbye!

(kali@kali)-[~]
$

```

Part 5: - Standard Test Table

TEST CASE	SCENARIO/OPERATION	INPUT	EXPECTED OUTPUT	RESULT
1 (VALID)	Addition Commutativity	Rows: 2 Columns: 2 A: 1 2, 3 4 B: 5 6, 7 8	(A + B) = 6 8, 10 12 (B + A) = 6 8, 10 12 "Addition IS Commutative"	PASS
2 (VALID)	Multiplication	N: 2	(A * B) = 19 22, 43 50	PASS

	Non-Commutativity	A: 1 2, 3 4 B: 5 6, 7 8	(B * A) = 23 34, 31 46 "Multiplication IS NOT Commutative"	
3 (BOUNDARY)	Multiplication Edge Case	N: 2 A: 0 1, 1 0 B: 1 0, 0 1	"Matrices accidentally matched." (Identity matrices are commutative).	PASS
4 (INVALID)	Invalid Dimensions	Addition-> Rows: -1	"Error: Dimensions must be greater than zero."	PASS

Part 6: - Evaluation of the Algorithm

The program takes heavy advantage of C99 Variable Length Arrays (VLAs), allowing dimensions *n*, *rows*, and *cols* to dictate array sizes at runtime without invoking slow heap allocations (*malloc*).